

Race Strategy Lab

From race data to a defensible decision

Backend MVP + model explanation
Prepared for Hemanth and the TrackShift team
12 September 2026

The central idea: receive a race snapshot, consider several possible rival responses, calculate the consequences of four actions, reject actions that breach configured limits, and explain the best remaining option.

This guide describes the code in the supplied ZIP exactly. The package name is a working label. The included `close_fight_s07` scenario is fictional; no real track or race session has been selected on your behalf.

Read this for	Pages
Data, libraries and model logic	2-6
Constraints and latency	7-8
Installation and API integration	9-10
Historical data and team workflow	11-12
Validation and judge answers	13-14
Limits, next steps and references	15-16

Scope: a runnable educational backend. The model uses demonstration coefficients and simulated energy. It is not validated for real race engineering, and no FIA regulation version is marked verified.

Where the data comes from

The model does not fetch data. An HTTP client downloads observations, validation creates a usable snapshot, and our own Python formulas calculate possible outcomes. The model produces estimates; it does not create missing measured facts.

Layer	This backend uses	What it actually does
Fetch	HTTPX	Calls the OpenF1 historical API and reads JSON.
Align	Python datetime + bisect	Matches the latest preceding observation; avoids future-data leakage.
Validate	Pydantic	Rejects invalid ranges, unknown fields and NaN/infinite values.
Calculate	NumPy + custom Python	Updates probability weights and calculates three futures per action.
Serve	FastAPI + Uvicorn	Accepts HTTP requests and returns structured decisions.
Remember	Python sqlite3	Stores each run, input, policy, belief and decision.
Verify	pytest + HTTPX	Checks calculations, API behavior and adapter contracts.

Pandas and SciPy are not required by this implementation. The earlier library list was a set of possible tools. This small model uses NumPy and standard Python; there is no benefit in claiming unused libraries.

Input data + explicit assumptions -> numerical model -> forecasts + constraints -> recommendation + reason. A future LLM could explain this structured output, but it is not part of the current backend.

Library roles follow the official HTTPX, NumPy and FastAPI documentation [3-5]. The equations themselves are our implementation assumptions, not supplied by those libraries.

The input contract

Input	Unit / meaning	Source in this release
own_speed_kph, rival_speed_kph	km/h; converted to m/s	Synthetic demo or OpenF1 car_data.
gap_s	Positive: rival ahead; negative: rival behind	Synthetic or numeric adjacent-driver interval.
own_energy_mj	MJ available to the scenario	Simulated or explicitly user supplied.
energy_uncertainty_mj	MJ downward allowance	User-supplied assumption; default 0.1.
recovery_kw	Assumed recovery power	Scenario input; default 10 kW.
recovery_uncertainty_kw	Downward recovery allowance	Scenario input; default 5 kW.
timestamp_s	Ordered run/replay clock	Synthetic seconds or historical UTC epoch.
data_age_s	Age of oldest required input	Caller value or adapter join age.
quality	0 to 1 input-quality score	Caller assigned; not calibrated confidence.
track_status	GREEN / YELLOW / RED / SC / VSC / UNKNOWN	Caller supplied; importer defaults UNKNOWN.

Source fields record whether speeds/gaps are synthetic, public, or supplied, and whether energy is simulated, supplied, or measured. These labels are caller declarations: the API does not independently authenticate a telemetry sensor.

Not used: throttle, brake, RPM, gear, DRS, weather, tyres and lap times. Public services may expose some of these, but this backend does not yet model their effects. Rival battery charge is never reconstructed from speed.

What “updating belief” means

We consider three hypotheses: the rival will have relatively slow, neutral, or fast pace. Initially each has weight 1/3. New relative-speed evidence changes those weights. It does not train a neural network or change response coefficients.

```
x = (rival_speed_kph - own_speed_kph) / 3.6
pace_means = [-2, 0, 2] m/s
q[j] = 0.9 * previous_weight[j] + 0.1 / 3
L[j] = exp(-0.5 * ((x - mean[j]) / 3)^2)
new_weight[j] = q[j] * L[j] / sum(q * L)
```

For a uniformly weighted prior and relative speed of +2 m/s, the updated slow/neutral/fast weights are approximately 0.186 / 0.362 / 0.452. Faster observed pace makes the fast hypothesis more plausible under this assumed noise model.

The 10% mixing toward a uniform prior prevents old evidence from creating permanent certainty. The implementation uses logarithms to avoid numerical underflow. These are fixed design choices, not statistically calibrated racing parameters.

What the weights do - and do not mean

A weight of 0.45 for “fast” means that scenario has 45% of the model weight. It does **not** mean a 45% chance of overtaking, a 45% battery level, or a validated prediction of the rival’s intentions. Corners, traffic and driver actions can all change observed speed.

The test suite demonstrates a meaningful effect: with the same current snapshot, a slow-heavy belief recommends ATTACK while a fast-heavy belief recommends HOLD. The result depends on our stated response assumptions.

How each possible future is calculated

Action	Power	Rival-response multiplier
ATTACK	120 kW	0.8
DEFEND	80 kW	0.4
HOLD	40 kW	0
CONSERVE	0 kW	0

For each action, evaluate slow, neutral and fast rival scenarios over 8 seconds in 0.5-second steps. All four power values and speed responses are demonstration assumptions.

```
own_future_mps = max(0, own_mps + 0.04 * (power_kw - 40))
rival_future_mps[j] = max(0, own_mps + pace_mean[j]
+ response_multiplier * [0, 1, 3][j])
distance[j] += (rival_future_mps[j] - own_future_mps) * dt
future_gap[j] = distance[j] / max(own_mps, 1)
```

The starting distance proxy is current gap multiplied by own speed. This is an approximate conversion; it is not a map-based measurement. The speed response jumps immediately to a constant value. There is no acceleration curve, corner map, drag, tyre or overtaking-geometry model.

```
E_next_MJ = min(capacity_MJ,
E_MJ + (recovery_kW - deploy_kW) * dt_s / 1000)
```

Why divide by 1,000? One kilowatt for one second is one kilojoule; one megajoule contains 1,000 kilojoules. The conservative calculation starts with lower energy and lower recovery, and checks the minimum energy at every step.

Counterfactual simply means “what would our model predict if we chose a different action?” These are conditional model outputs, not observed alternative race outcomes.

A worked example from the actual code

Input: own speed 300 km/h, rival speed 302 km/h, rival 0.7 s ahead, energy 2.4 MJ, confirmed demo GREEN. Default uncertainty is 0.1 MJ, recovery is 10 kW with a 5 kW allowance, and reserve is 0.6 MJ.

```
ATTACK expected energy = 2.4 - (120 - 10) * 8 / 1000
= 1.520 MJ
ATTACK conservative energy = 2.4 - 0.1 - (120 - 5) * 8 / 1000
= 1.380 MJ
```

Action	Energy MJ	Gap proxy s	Score s	Valid
ATTACK	1.520	0.517	-0.070	Yes
DEFEND	1.840	0.616	-0.098	No
HOLD	2.160	0.715	-0.126	Yes
CONSERVE	2.480	0.868	-0.231	Yes

```
progress = current_gap - weighted_future_gap
score = progress - 0.15 * energy_used_MJ - 0.5 * gap_spread
```

The score converts energy use into a seconds-equivalent cost using an explicit weight. The gap spread penalizes disagreement between the three scenarios. A higher score is preferred only after constraints pass.

Actual recommendation in this example: ATTACK. DEFEND is rejected because the rival is ahead, not behind. The exact JSON is included as docs/example_decision.json.

A predicted smaller gap does not prove a completed pass. Do not describe progress_s as an actual lap-time saving. We also do not output a made-up “78% confidence” score.

When the backend refuses to advise

Condition	Behavior
Data older than max_data_age_s	Abstain; do not update belief.
Quality below min_quality	Abstain; do not update belief.
Track status not GREEN	Abstain; request engineer review.
Own speed below 30 km/h	Abstain outside supported regime.
Energy above configured capacity	Abstain on inconsistent state.
Action exceeds deployment cap	Reject that action.
Conservative energy crosses reserve	Reject that action, including initial reserve breach.
ATTACK with no nearby rival ahead	Reject ATTACK.
DEFEND with no nearby rival behind	Reject DEFEND.
No valid action remains	Return NO_RECOMMENDATION.

Default proximity windows are 1.5 seconds; default reserve is 0.6 MJ. They are **model policy**, not claims about FIA regulations. The API reports that rules have not been verified.

Among valid actions, choose the highest score. If the top two are nearly tied and HOLD is valid, choose HOLD to avoid unnecessary switching. However, HOLD itself is not automatically safe: it uses energy in this model.

We deliberately return NO_RECOMMENDATION when the input or constraints cannot support a decision. “Always hold the current strategy” could preserve an already invalid action. The engineer remains responsible for interpreting the situation.

Latency: what we actually measured

A fast model cannot make delayed source data fresh. Separate source delivery time, numerical processing, API/storage work, network delivery, and human communication.

Measured path	N	p50 ms	p95 ms	p99 ms
Engine only	1000	0.312	0.335	0.358
In-process API	200	1.418	1.785	2.418
API + SQLite	200	1.817	2.422	3.109

Measured on Python 3.12.14 / Linux x86_64. The benchmark uses one client, a mixed set of synthetic valid and abstaining snapshots, and 50 engine warmups. No external telemetry delivery is included. Full metadata and maxima are in docs/benchmark.json.

p95 means 95% of measured calls finished at or below that duration. The API measurements use TestClient inside the process; they are not full network round trips. A separate real localhost HTTP smoke check passed, but is not used as a racing-latency benchmark.

Why it is fast

Only four actions and three pace scenarios are evaluated over a bounded horizon. No model download or LLM call occurs in the decision path. External historical downloads run in a separate CLI tool.

The earlier 150 ms figure was a design target. These measurements support fast local processing on this machine; they do not guarantee worst-case deadlines on your laptop or end-to-end live F1 latency. Rerun the included benchmark on the demonstration computer.

OpenF1 documents public car-data sampling at approximately 3.7 Hz [1]. That source cadence is separate from the engine's execution speed.

Start the backend on Windows 11

Use Python 3.12. Extract the ZIP, open the race-strategy-backend folder in VS Code, and open a terminal there. The current directory should contain README.md and requirements.txt.

Terminal 1: install and run

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Keep this terminal running. Open **<http://127.0.0.1:8000/docs>** for interactive API documentation. Swagger UI loads browser assets from a CDN; the underlying API works offline after dependencies are installed.

Terminal 2: demonstrate and test

```
.\.venv\Scripts\python.exe -m scripts.demo
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m scripts.benchmark
```

The demo creates a stored run, submits 12 ordered synthetic snapshots, prints each recommendation, and writes data/latest_decisions.csv. The snapshots exercise low energy, caution flags, stale data and a rival behind. They are illustrative cases, not a continuous physically simulated race.

Common setup problems

Problem	Fix
No py command	Install Python 3.12 and reopen the terminal.
Module not found	Use the .venv Python executable and install requirements.
Port 8000 is occupied	Use --port 8001 and demo --url http://127.0.0.1:8001.
401 Unauthorized	Set the same API_KEY in the server and client terminals.
409 Conflict	Timestamp was repeated or older; retrieve stored history.

API workflow for your frontend team

Use a persisted run so the backend carries the belief forward. Each run has a fixed policy. A new timestamp updates the belief and inserts the input/output record in one SQLite transaction.

1. POST /v1/runs body: {"name":"team-demo"}
2. Save the returned id
3. POST /v1/runs/{id}/steps body: one State snapshot
4. GET /v1/runs/{id}/decisions retrieve input + output audit
5. GET /v1/runs/{id}/export.csv download summary

Minimal synthetic State body

```
{
  "timestamp_s": 1,
  "own_speed_kph": 300,
  "rival_speed_kph": 302,
  "gap_s": 0.7,
  "own_energy_mj": 2.4,
  "track_status": "GREEN"
}
```

The response contains recommendation, status, reason, candidate scores and rejected constraints, updated belief, score margin, engine latency and provenance. Show model assumptions beside the recommendation in your frontend.

POST /v1/decide is stateless: send a prior belief explicitly if you want continuity. POST /v1/replay carries belief across an ordered list of up to 2,000 frames, but does not persist it or alter future recorded observations.

History and CSV endpoints accept limit and offset. Timestamps must increase strictly in each run. Simultaneous duplicate submissions commit only once; the other receives 409. If a response was lost, retrieve history before resubmitting a newer timestamp.

Read docs/openapi.json for all fields and ranges. CORS origins and the optional X-API-Key are configurable. Local Python does not auto-load .env; see README.md for exact environment commands.

Bring in a real historical window

First choose a real session and two adjacent drivers at the same time. Your project’s circuit has not been confirmed in this conversation; do not present the fictional demo as a particular track.

OpenF1 documents historical access without authentication and separate requirements for live data [1]. The importer is designed for short historical windows. It fetches car_data, position and intervals for the two drivers, preserves raw JSON, then creates replay snapshots.

```
python -m scripts.fetch_openf1 --session SESSION_KEY
--own OWN_NUMBER --rival RIVAL_NUMBER
--start ISO_START_WITH_TIMEZONE --end ISO_END_WITH_TIMEZONE
--energy-mj 2.4 --output data/openf1_replay.json
```

Put this command on **one line** and replace every placeholder. Keep the window at most five minutes. Then run `python -m scripts.demo --file data/openf1_replay.json`.

Adapter behavior	Why it matters
Only past samples are joined	No future information leaks into a decision.
Drivers must be adjacent	A public interval is to the car ahead, not an arbitrary opponent.
Non-numeric/lapped intervals are skipped	A lap deficit is not treated as a time gap.
Energy stays simulated	Public speed is not measured battery charge.
UNKNOWN track status by default	Public flag interpretation is not automated here.

Only add `--assume-green` for an explicitly labelled demonstration assumption. Position freshness is included conservatively; the adapter may abstain frequently when position updates are old.

Delivery limitation: a live HTTP smoke request returned HTTP 404, so a real-data download was not verified end to end here. Adapter behavior was tested with HTTPX mocks. No real race replay is bundled, and no synthetic fallback is disguised as a successful download.

How the team should work with this ZIP

File / area	Ownership suggestion	What to explain
app/engine.py	Gomathi / Hemanth	Belief update, action futures, energy and scoring.
app/openf1.py + fetch script	Data owner	Source fields, time alignment and missing data.
app/main.py + schemas.py	Backend / integration owner	Request contract and frontend integration.
app/storage.py	Backend owner	Persistent audit and atomic updates.
tests + benchmark	Validation owner	What passed, what was measured, what remains unproven.
README + this guide	Every presenter	Setup, assumptions and honest judge answers.

A practical 5-minute team walkthrough

Minute 1: show the problem: spending energy now can affect the next opportunity. Explain the public-data and simulated-energy boundary.

Minute 2: send the ordinary GREEN snapshot. Open the candidate table and explain one energy calculation in MJ.

Minute 3: reduce energy. Show ATTACK rejected. Then send YELLOW or stale data and show abstention.

Minute 4: show the stored belief and demonstrate the belief-sensitive unit test. Explain why a neural network is not necessary for this planner.

Minute 5: show CSV history, the test report and p95 latency. State that prediction accuracy still requires simulator or team data.

Do not use real driver names to imply that the synthetic scenario belongs to them. For a selected real replay, record the session, circuit, drivers, time window, source query and any manual assumptions together.

What has been tested

Validation	Outcome / scope
Automated tests	30 passed. Numerical units, reserve behavior, belief adaptation, gates, API, persistence, duplicate concurrency, data alignment and HTTP errors.
Generated input coverage	A test checks 200 seeded energy/recovery combinations for reserve compliance of selected actions.
Real localhost HTTP flow	Uvicorn health -> create run -> 12 steps -> history -> CSV passed.
Determinism	Same state, policy and belief yields same output except measured latency.
OpenF1 adapter	Mocked contract tests passed; live historical smoke returned 404.
Docker / Windows	Instructions and files provided; neither environment was executed here.
Race prediction accuracy	Not validated. There is no measured claim of race benefit.

The complete test output is in docs/test-results.txt. It contains two upstream dependency deprecation warnings; tests still pass. requirements.txt pins direct runtime dependencies, while requirements-dev.txt adds pytest.

What a passing test establishes

A correct MJ calculation establishes that the formula was implemented correctly. It does not establish that a chosen car actually deploys the assumed power. A reserve test establishes compliance with the configured model policy, not with an unseen regulation or sensor error.

Do not turn a software success metric into a race-performance claim. “30 tests passed” is supported. “The system improves overtaking by 20%” is not supported by this package.

Judge questions: answers you can defend

“Where is the AI if there is no trained model?”

“We use a model-based decision planner with a Bayesian pace belief. It compares action-dependent futures and updates scenario weights from observations. We do not claim a neural network. The most precise description is an adaptive numerical decision-support MVP.”

“Is it just a rule engine?”

“Rules enforce configured limits. Numerical simulation and weighted outcomes rank the valid actions. We can inspect both separately. Our tests show the recommended action can change when the pace belief changes.”

“Where did those power and response values come from?”

“They are explicit demonstration assumptions, not learned or verified car parameters. We expose them so reviewers can inspect them. Simulator or team telemetry is needed to calibrate and validate them.”

“Can you know the rival’s remaining battery?”

“No. We do not claim that public speed reveals private battery telemetry. We model uncertain pace scenarios and keep energy inputs explicitly supplied or simulated.”

“Does it learn, and why not an LLM?”

“It updates scenario probabilities within a run; it does not retrain parameters. The critical path needs reproducible numerical calculations. An LLM could later explain the output, but cannot supply missing measurements.”

“How fast and accurate is it?”

“We measured local processing distributions and include the benchmark. Those numbers exclude source delay. Predictive accuracy and racing benefit have not yet been validated; that is a separate experiment.”

“Can you prove the alternative action would have worked?”

“Only inside the stated model. A historical replay observes one actual future. Controlled closed-loop simulation is needed to compare alternative actions fairly.”

Next steps toward credible racing results

The backend is complete for its declared MVP scope. Improving the quality of the racing advice requires better evidence and a richer model, rather than simply adding a larger AI library.

Next step	Evidence to collect
Choose the actual circuit/session	Confirmed session key, drivers and fight window.
Build a controlled simulator	Known state, executed actions and resulting energy/gaps.
Calibrate energy and speed response	Observed response to deployment under controlled conditions.
Use a held-out evaluation set	Gap MAE/RMSE and energy error on unseen scenarios.
Compare against simple baselines	Always-HOLD and a reserve rule under the same simulator seeds.
Validate uncertainty	How often predicted intervals contain observed outcomes.
Verify regulations explicitly	Exact document, issue, year, clause and implementation tests.
Measure the real integration	Source age, network time, concurrent load and decision deadlines.

Add tyre, fuel, corner and overtaking geometry models only when inputs and validation data are available. The present constant-speed approximation cannot distinguish a straight from a corner or determine whether two cars can pass safely.

For an externally hosted multi-user service, add proper identity and authorization, TLS, request-size and rate limits, operational monitoring and a stronger concurrency design. The supplied optional API key and SQLite are appropriate for a local team demonstration, not a full team platform.

A good project claim is specific: “We built and tested an explainable, uncertainty-aware planning backend, with explicit assumptions and auditable decisions.” Your validation work can then earn stronger claims.

References and delivery checklist

Primary documentation checked on 12 September 2026. Sources describe data interfaces and software tools; none of them validates our custom racing dynamics.

[1] **OpenF1 API documentation**

<https://openf1.org/docs/>

[2] **OpenF1 service overview**

<https://openf1.org/>

[3] **HTTPX QuickStart**

<https://www.python-httpx.org/quickstart/>

[4] **NumPy: What is NumPy?**

<https://numpy.org/doc/stable/user/whatisnumpy.html>

[5] **FastAPI Tutorial**

<https://fastapi.tiangolo.com/tutorial/>

Files to open first

File	Purpose
README.md	Exact install, run, API and troubleshooting steps.
docs/DATA_AND_MODEL.md	Complete equations and assumptions in editable text.
docs/example_decision.json	Real output generated by this backend.
docs/openapi.json	Frontend request/response contract.
docs/benchmark.json	Measured timings and environment.
docs/test-results.txt	Full automated test result.
docs/Team_Explanation.pdf	This handbook, also included inside the ZIP.

Before presenting: run the demo on your laptop, read the candidate table, know the MJ calculation, show the synthetic labels, and agree who answers model, data and validation questions.

Deliverables: source ZIP and this PDF. No private F1 data, production credentials, trained model, or real race-performance guarantee is included.